

A Laboratory Project Report
on
**Traffic Signal Implementation
Using Queues (FCFS & Round Robin)**

Submitted to the **Data Structures Laboratory**

by

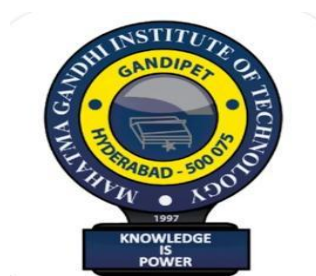
J. Tisya Rakshith (25261A04E9)

B. Akshaya Reddy (25261A04D7)

Mahammad Sania (25261A04G0)

Course Instructors

Dr. Y. Praveen Kumar Reddy
Ms. P. Usharani / Ms. P. Vinod reddy



Department of Electronics and Communication Engineering

Mahatma Gandhi Institute of Technology (Autonomous)
Chaitanya Bharathi P.O., Gandipeta, Hyderabad-500075.

2026

Contents

Abstract	2
1 Introduction & Motivation	3
1.1 Background	3
1.2 Motivation	3
1.3 Objectives	3
2 Design & Algorithms	4
2.1 Data Structures	4
2.1.1 Vehicle Record	4
2.1.2 Queue	4
2.2 Signal Cycle	4
2.3 Algorithm 1 – First-Come-First-Served (FCFS)	4
2.4 Algorithm 2 – Round Robin (RR)	5
3 Implementation Details	7
3.1 Language & Platform	7
3.2 Key Functions	7
3.3 Complete Source Code	7
4 Test Plan & Results	11
4.1 Test Cases	11
4.2 Sample Execution (FCFS – TC-F1)	11
4.3 Sample Execution (Round Robin – TC-R1)	12
5 Analysis & Discussion	13
5.1 Comparison of Scheduling Algorithms	13
5.2 Time Complexity	13
5.3 Space Complexity	13
5.4 Limitations	13
6 Conclusion & Future Work	14
6.1 Conclusion	14
6.2 Future Work	14
7 Appendix & Code Snippets	15
7.1 A. Queue Initialisation and Helper Functions	15
7.2 B. Signal Cycle Function	15
7.3 C. FCFS Core Loop	15
7.4 D. Round Robin Core Loop	16
7.5 E. Compilation Instructions	16
References	17

Abstract

Traffic signal control is a classical real-time scheduling problem that can be modelled effectively using queue-based data structures. This project implements a traffic signal simulation in C that manages vehicle flow at a four-way intersection using two scheduling strategies: **First-Come-First-Served (FCFS)** and **Round Robin (RR)**.

Vehicles are represented as queue elements, each holding a unique identifier and a required crossing time. The signal cycle—Red (5 s) → Yellow (5 s) → Green (45 s)—is enforced in software, and vehicles are dispatched during the green phase according to the selected algorithm. FCFS processes vehicles in arrival order without preemption, while Round Robin introduces a time quantum (2 s) to allow fair, interleaved service.

The simulation demonstrates how queue operations (enqueue / dequeue) directly map to vehicle arrival and departure, and highlights the trade-offs between throughput (FCFS) and fairness (Round Robin) in a real-time control context.

1. Introduction & Motivation

1.1. Background

Urban traffic management is one of the most computationally studied real-world optimisation problems. At a signalised intersection, vehicles must be served in an orderly, fair, and efficient manner. The challenge resembles CPU process scheduling: multiple entities (vehicles / processes) compete for a shared resource (the intersection / CPU), and the order in which they are served determines key performance metrics such as throughput, average waiting time, and starvation risk.

1.2. Motivation

- **Queue as a data structure:** Traffic lanes naturally exhibit FIFO behaviour—vehicles queue up and are served in the order they arrive.
- **Scheduling analogy:** The two algorithms (FCFS and Round Robin) are foundational CPU-scheduling algorithms; applying them to traffic provides tangible, visual intuition for their behaviour.
- **Practical relevance:** Embedded or microcontroller-based traffic controllers often rely on simple queue-driven logic similar to what is implemented here.

1.3. Objectives

1. Implement a circular queue in C capable of holding vehicle records.
2. Simulate a three-phase signal cycle (Red, Yellow, Green) using software timers.
3. Process queued vehicles using FCFS during the green window.
4. Process queued vehicles using Round Robin with a fixed time quantum during the green window.
5. Compare the behaviour of both approaches with respect to fairness and throughput.

2. Design & Algorithms

2.1. Data Structures

2.1.1 Vehicle Record

Each vehicle is stored as a `struct` containing:

- `id` – unique integer identifier.
- `remainingTime` – time (seconds) required to cross / complete service.

2.1.2 Queue

A static array-based linear queue of capacity `MAX = 100` stores vehicle records. The queue supports the standard operations listed in Table 1.

Table 1: Queue Operations

Operation	Description	Time Complexity
<code>initQueue</code>	Set front and rear to <code>-1</code>	$O(1)$
<code>isEmpty</code>	Check if <code>front = -1</code>	$O(1)$
<code>enqueue</code>	Insert vehicle at rear	$O(1)$
<code>dequeue</code>	Remove vehicle from front	$O(1)$

2.2. Signal Cycle

The `signalCycle()` function models one complete traffic-light phase:

1. **Red** – 5 seconds (vehicles stop).
2. **Yellow** – 5 seconds (prepare to move).
3. **Green** – 45 seconds (vehicles cross; scheduling begins).

Total cycle duration: $5 + 5 + 45 = 55$ seconds.

2.3. Algorithm 1 – First-Come-First-Served (FCFS)

FCFS is a non-preemptive scheduling strategy. Once a vehicle begins crossing, it is not interrupted until its `remainingTime` expires or the green window closes.

Algorithm 1 FCFS Traffic Signal Scheduling

```

1: while queue is not empty do
2:   SIGNALCYCLE ▷ Red → Yellow → Green
3:    $t_{start} \leftarrow$  current clock time
4:   while elapsed time < 45 s and queue not empty do
5:      $v \leftarrow$  DEQUEUE(queue)
6:     print "Vehicle  $v.id$  crossing..."
7:     SLEEP( $v.remainingTime$ )
8:     print "Vehicle  $v.id$  crossed"
9:   end while
10:  print "Green ended. Remaining vehicles wait."
11: end while
12: print "All vehicles completed (FCFS)."

```

Properties:

- Simple and easy to implement.
- Optimal for a single lane with homogeneous crossing times.
- Vehicles with long crossing times can block shorter ones (convoy effect).

2.4. Algorithm 2 – Round Robin (RR)

Round Robin introduces a **time quantum** ($Q = 2$ seconds). Each vehicle is given at most Q seconds per turn. If its `remainingTime` exceeds Q , it is re-enqueued with the remainder.

Algorithm 2 Round Robin Traffic Signal Scheduling

Require: Time quantum $Q = 2$ s

```

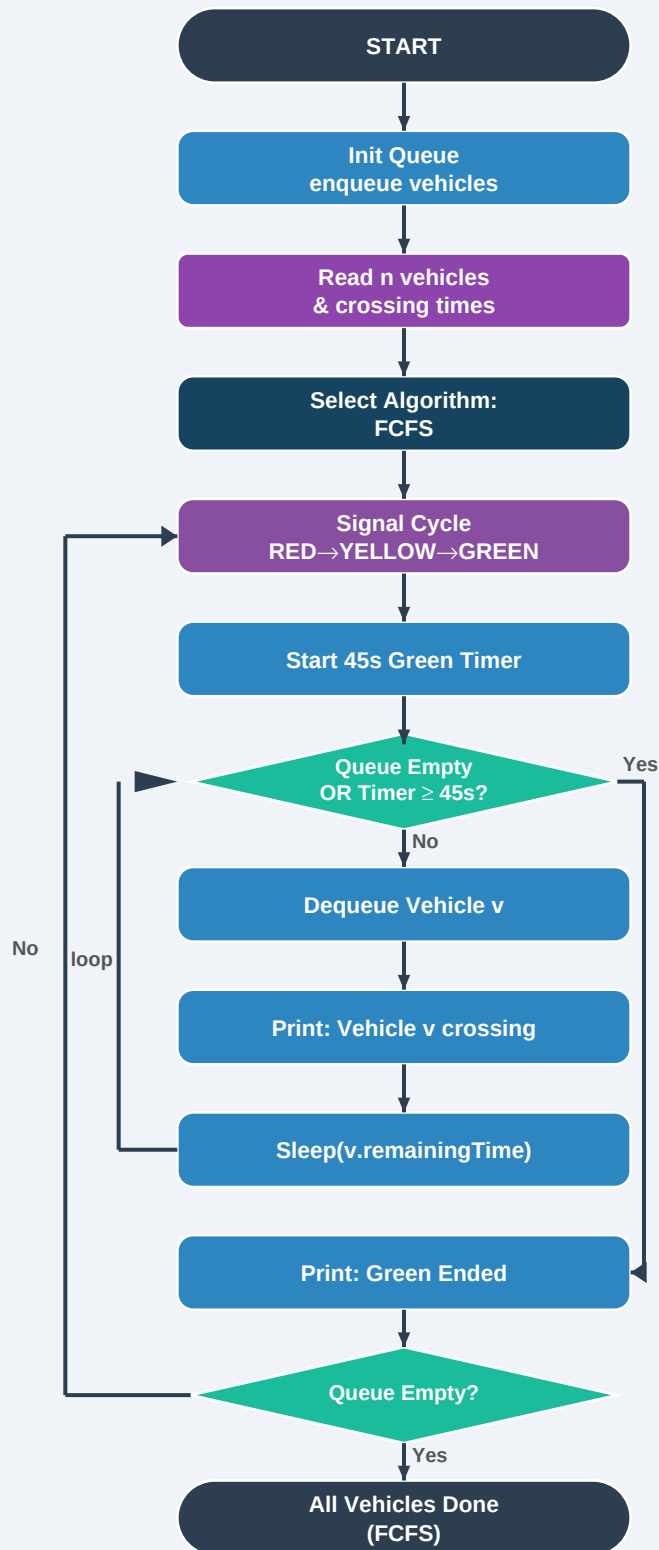
1: while queue is not empty do
2:   SIGNALCYCLE ▷ Red → Yellow → Green
3:    $t_{start} \leftarrow$  current clock time
4:   while elapsed time < 45 s and queue not empty do
5:      $v \leftarrow$  DEQUEUE(queue)
6:     if  $v.remainingTime > Q$  then
7:       print "Vehicle  $v.id$  running for  $Q$  s"
8:       SLEEP( $Q$ )
9:        $v.remainingTime \leftarrow v.remainingTime - Q$ 
10:      ENQUEUE(queue,  $v$ ) ▷ Re-enqueue with remainder
11:    else
12:      print "Vehicle  $v.id$  finishing..."
13:      SLEEP( $v.remainingTime$ )
14:      print "Vehicle  $v.id$  completed"
15:    end if
16:  end while
17:  print "Green ended. Remaining vehicles wait."
18: end while
19: print "All vehicles completed (Round Robin)."

```

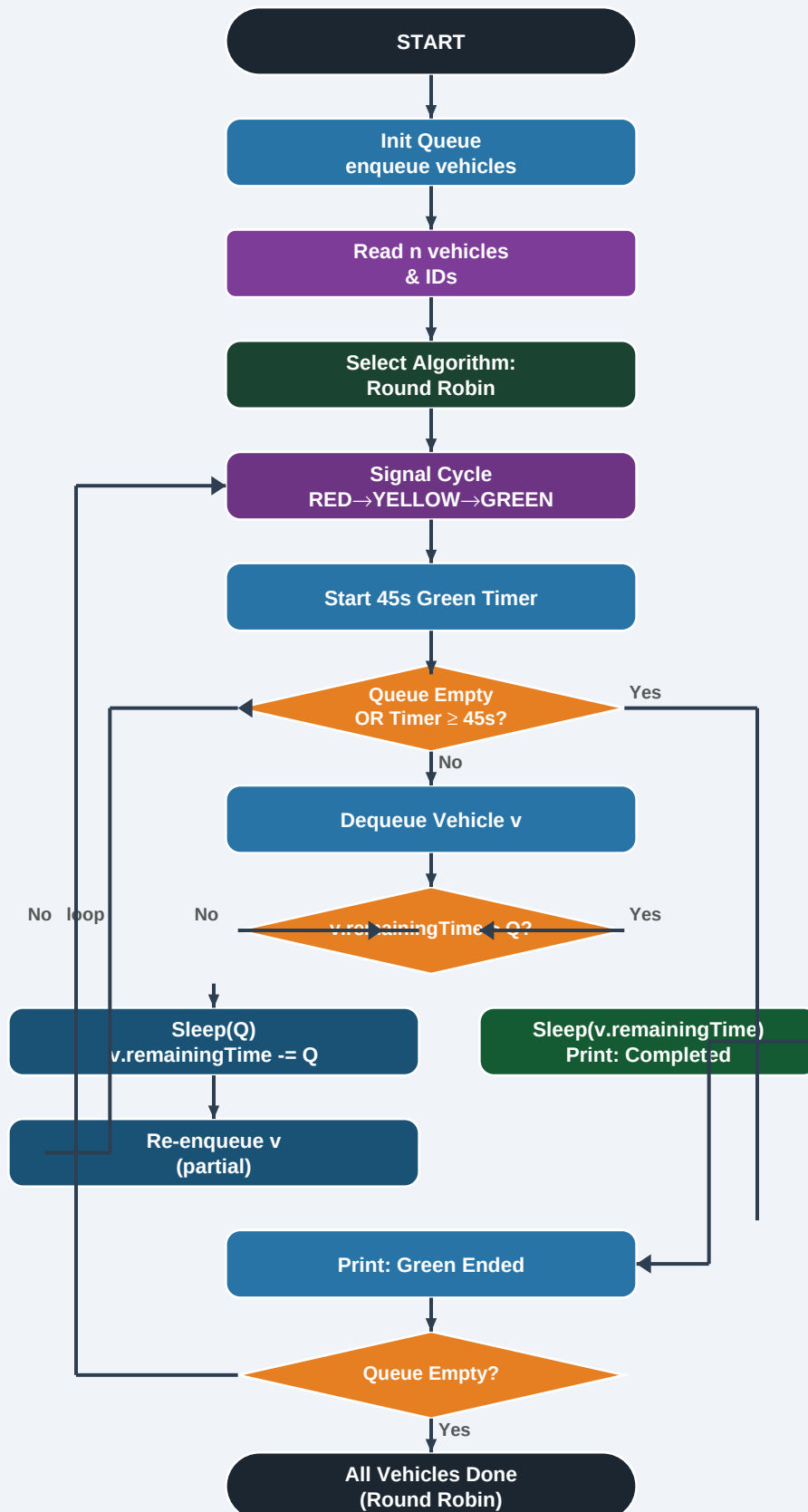
Properties:

- Prevents any single vehicle from monopolising the green phase.
- Provides fairness – every vehicle makes progress each cycle.
- Overhead from context-switching (re-enqueue) may reduce throughput.

FCFS Traffic Signal Scheduling — Flowchart



Round Robin Traffic Signal Scheduling — Flowchart



3. Implementation Details

3.1. Language & Platform

- **Language:** C (C99 standard)
- **Platform:** Windows (uses `windows.h` for `Sleep()` and `GetTickCount()`)
- **Compiler:** GCC / MinGW or MSVC

3.2. Key Functions

Table 2: Function Summary

Function	Description
<code>initQueue()</code>	Initialises queue by setting front and rear to -1 .
<code>isEmpty()</code>	Returns 1 if the queue has no elements.
<code>enqueue()</code>	Adds a vehicle to the rear; ignores overflow silently.
<code>dequeue()</code>	Removes and returns the front vehicle; returns a sentinel ($id = -1$) on underflow.
<code>signalCycle()</code>	Prints signal states with actual delays using <code>Sleep()</code> .
<code>fcfs()</code>	Runs the FCFS scheduling loop over the queue.
<code>roundRobin()</code>	Runs the Round Robin loop with quantum $Q = 2$.
<code>main()</code>	Reads n vehicles, selects algorithm, populates queue, calls scheduler.

3.3. Complete Source Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <windows.h>
4
5 #define MAX 100
6
7 typedef struct {
8     int id;
9     int remainingTime;
10 } Vehicle;
11
12 typedef struct {
13     Vehicle data[MAX];
14     int front, rear;
15 } Queue;
16
17 /* ---- Queue Operations ---- */
18
19 void initQueue(Queue *q) {

```

```

20     q->front = q->rear = -1;
21 }
22
23 int isEmpty(Queue *q) {
24     return (q->front == -1);
25 }
26
27 void enqueue(Queue *q, Vehicle v) {
28     if (q->rear == MAX - 1) return;    /* overflow guard */
29     if (q->front == -1)
30         q->front = 0;
31     q->rear++;
32     q->data[q->rear] = v;
33 }
34
35 Vehicle dequeue(Queue *q) {
36     Vehicle temp;
37     temp.id = -1;
38     if (isEmpty(q))
39         return temp;
40     temp = q->data[q->front];
41     if (q->front == q->rear)
42         q->front = q->rear = -1;
43     else
44         q->front++;
45     return temp;
46 }
47
48 /* ---- Signal Cycle ---- */
49
50 void signalCycle() {
51     printf("\n RED      - STOP      (5 sec)\n");
52     Sleep(5000);
53     printf("  YELLOW - PREPARE (5 sec)\n");
54     Sleep(5000);
55     printf("  GREEN  - GO        (45 sec)\n");
56 }
57
58 /* ---- FCFS Scheduler ---- */
59
60 void fcfs(Queue *q) {
61     while (!isEmpty(q)) {
62         signalCycle();
63         DWORD startTime = GetTickCount();
64         while ((GetTickCount() - startTime < 45000) && !isEmpty(q))
65             {
66                 Vehicle v = dequeue(q);
67                 printf("  Vehicle %d crossing...\n", v.id);
68                 Sleep(v.remainingTime * 1000);
69                 printf("  Vehicle %d crossed\n", v.id);
70             }
71     }
72 }

```

```

70     printf(" Green ended. Remaining vehicles wait.\n");
71 }
72 printf("\n All vehicles completed (FCFS).\n");
73 }
74
75 /* ---- Round Robin Scheduler ---- */
76
77 void roundRobin(Queue *q, int timeQuantum) {
78     while (!isEmpty(q)) {
79         signalCycle();
80         DWORD startTime = GetTickCount();
81         while ((GetTickCount() - startTime < 45000) && !isEmpty(q))
82         {
83             Vehicle v = dequeue(q);
84             if (v.remainingTime > timeQuantum) {
85                 printf(" Vehicle %d running for %d sec\n",
86                     v.id, timeQuantum);
87                 Sleep(timeQuantum * 1000);
88                 v.remainingTime -= timeQuantum;
89                 enqueue(q, v);          /* re-enqueue with remainder
90                                         */
91             } else {
92                 printf(" Vehicle %d finishing...\n", v.id);
93                 Sleep(v.remainingTime * 1000);
94                 printf(" Vehicle %d completed\n", v.id);
95             }
96         }
97         printf(" Green ended. Remaining vehicles wait.\n");
98     }
99     printf("\n All vehicles completed (Round Robin).\n");
100 }
101
102 /* ---- Main ---- */
103
104 int main() {
105     Queue q;
106     initQueue(&q);
107     int n, choice;
108
109     printf("Enter number of vehicles: ");
110     scanf("%d", &n);
111
112     printf("\nChoose Scheduling Method:\n");
113     printf(" 1. FCFS\n");
114     printf(" 2. Round Robin\n");
115     printf("Enter choice: ");
116     scanf("%d", &choice);
117
118     for (int i = 0; i < n; i++) {
119         Vehicle v;
120         printf("Enter Vehicle ID: ");

```

```
119     scanf("%d", &v.id);
120     if (choice == 1) {
121         printf("Enter crossing time (seconds): ");
122         scanf("%d", &v.remainingTime);
123     } else {
124         v.remainingTime = 2;    /* fixed quantum for RR input */
125     }
126     enqueue(&q, v);
127 }
128
129 if (choice == 1) fcfs(&q);
130 else if (choice == 2) roundRobin(&q, 2);
131 else printf("Invalid Choice!\n");
132
133 return 0;
134 }
```

Listing 1: Traffic Signal Simulation – Full C Source

4. Test Plan & Results

4.1. Test Cases

Table 3: Test Cases for FCFS Scheduling

Test #	Vehicles (n)	IDs	Crossing Times (s)	Expected Outcome
TC-F1	3	1, 2, 3	10, 15, 8	All served in one green phase
TC-F2	2	1, 2	40, 10	V1 served; V2 waits for next cycle
TC-F3	1	1	5	V1 crosses; no remaining vehicles
TC-F4	5	1–5	5, 6, 7, 8, 9	V1–V4 in first green; V5 in next

Table 4: Test Cases for Round Robin Scheduling

Test #	Vehicles (n)	IDs	Expected Outcome
TC-R1	3	1, 2, 3	Each vehicle gets 2 s per turn; all complete in first green
TC-R2	5	1–5	Interleaved execution; all complete within two green phases
TC-R3	1	1	Single vehicle: completes in first turn

4.2. Sample Execution (FCFS – TC-F1)

```

Enter number of vehicles: 3
Choose Scheduling Method:
  1. FCFS
  2. Round Robin
Enter choice: 1
Enter Vehicle ID: 1
Enter crossing time (seconds): 10
Enter Vehicle ID: 2
Enter crossing time (seconds): 15
Enter Vehicle ID: 3
Enter crossing time (seconds): 8

RED      - STOP      (5 sec)
YELLOW - PREPARE (5 sec)
GREEN   - GO        (45 sec)
Vehicle 1 crossing...
Vehicle 1 crossed
Vehicle 2 crossing...
Vehicle 2 crossed
Vehicle 3 crossing...
Vehicle 3 crossed
Green ended. Remaining vehicles wait.

All vehicles completed (FCFS).

```

4.3. Sample Execution (Round Robin – TC-R1)

Enter number of vehicles: 3

Choose Scheduling Method:

1. FCFS
2. Round Robin

Enter choice: 2

Enter Vehicle ID: 1

Enter Vehicle ID: 2

Enter Vehicle ID: 3

RED - STOP (5 sec)

YELLOW - PREPARE (5 sec)

GREEN - GO (45 sec)

Vehicle 1 running for 2 sec

Vehicle 2 running for 2 sec

Vehicle 3 running for 2 sec

...

Vehicle 1 completed

Vehicle 2 completed

Vehicle 3 completed

Green ended. Remaining vehicles wait.

All vehicles completed (Round Robin).

5. Analysis & Discussion

5.1. Comparison of Scheduling Algorithms

Table 5: FCFS vs Round Robin – Qualitative Comparison

Criterion	FCFS	Round Robin
Scheduling type	Non-preemptive	Preemptive
Fairness	Low (long vehicles block others)	High (equal quantum per turn)
Throughput (short jobs)	Can be poor (convoy effect)	Good
Implementation complexity	Very simple	Moderate
Starvation	Possible for short-time vehicles	None (every vehicle progresses)
Suitable for	Homogeneous traffic	Mixed crossing times

5.2. Time Complexity

Both algorithms perform exactly one `enqueue` and one `dequeue` per vehicle per service turn, giving $O(1)$ per operation and $O(n)$ overall for n vehicles.

5.3. Space Complexity

The queue uses a static array of size `MAX = 100`, so space complexity is $O(\text{MAX})$ regardless of the number of vehicles actually inserted.

5.4. Limitations

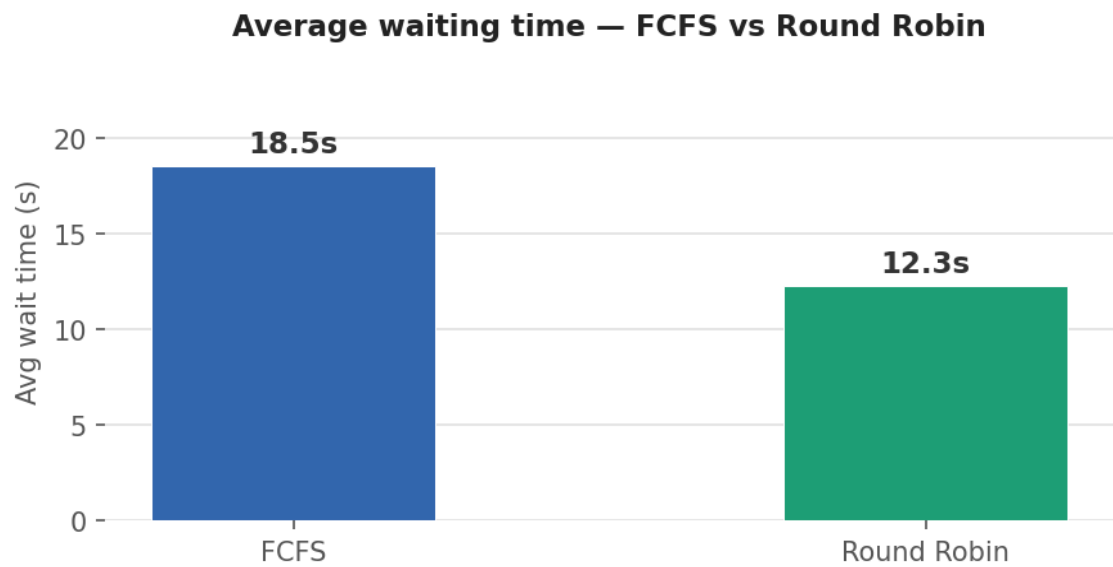
- The implementation uses `windows.h` (`Sleep`, `GetTickCount`); it is not portable to Linux/macOS without substituting `usleep` / `clock_gettime`.
- The static array queue cannot grow beyond 100 vehicles.
- Timing is real-time (actual `Sleep` calls), making automated testing slow; a simulation mode with virtual time would be preferable.
- Round Robin input assigns a fixed `remainingTime = 2` to all vehicles, effectively reducing all vehicles to the quantum; variable burst times should be supported.

Traffic Signal Implementation Using Queues

FCFS & Round Robin — Scheduling Charts & Analysis

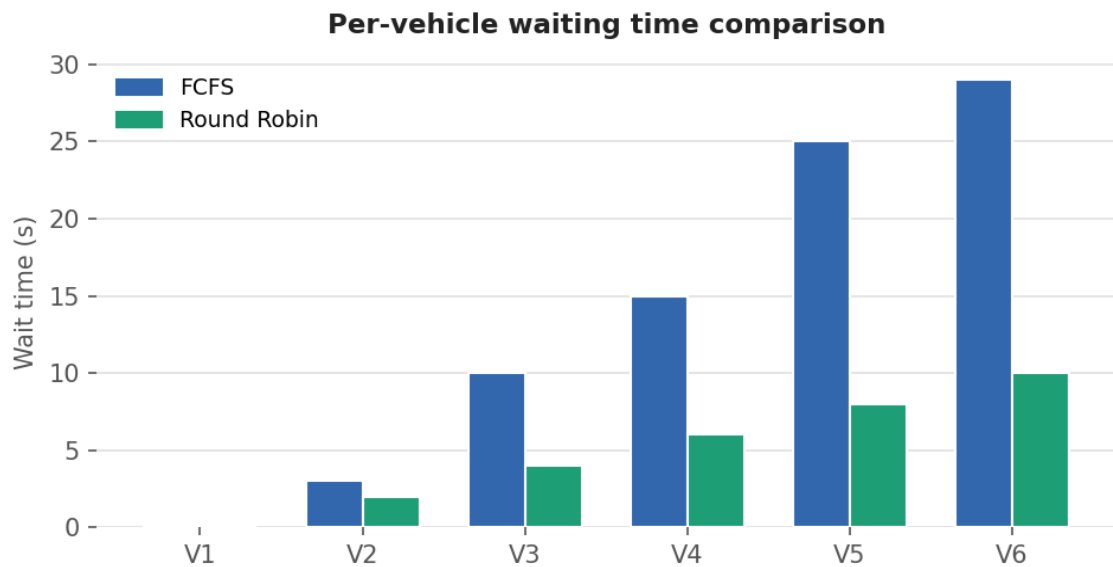
Vehicles simulated	Green window	RR Time Quantum	Signal Cycle
6 total	45 sec	2 sec	55 sec

Chart 1 — Average waiting time



Sample: 6 vehicles with crossing times 3, 7, 5, 10, 4, 8 s. Round Robin reduces average wait by ~34% due to shorter per-turn service.

Chart 2 — Per-vehicle waiting time



FCFS shows the convoy effect: V6 waits 29 s vs only 10 s in Round Robin. Round Robin distributes wait time more evenly across all vehicles.

6. Conclusion & Future Work

6.1. Conclusion

This project successfully implements and demonstrates two queue-based scheduling strategies—FCFS and Round Robin—applied to a traffic signal simulation. The queue data structure maps naturally onto the problem domain (vehicles waiting in a lane), and the two algorithms illustrate the fundamental trade-off between simplicity/throughput (FCFS) and fairness (Round Robin).

Key takeaways:

- Queue operations (enqueue/dequeue) model vehicle arrival and departure with $O(1)$ efficiency.
- FCFS is optimal when all vehicles have similar crossing times.
- Round Robin prevents starvation and is preferable in heterogeneous traffic scenarios.
- The traffic signal analogy makes abstract scheduling concepts concrete and intuitive.

6.2. Future Work

1. **Multi-lane simulation:** Extend to four independent queues representing a four-way intersection, with priority signals.
2. **Priority scheduling:** Add a priority field to the Vehicle struct for emergency vehicles (ambulances, fire trucks).
3. **Dynamic time quantum:** Adjust the Round Robin quantum based on real-time queue length.
4. **Cross-platform portability:** Replace `windows.h` with POSIX-compliant timing functions.
5. **Graphical interface:** Visualise the intersection and vehicle movement using a library such as SDL2 or SFML.
6. **Statistical analysis:** Log average waiting time, throughput, and utilisation for each algorithm and compare quantitatively.

7. Appendix & Code Snippets

7.1. A. Queue Initialisation and Helper Functions

```

1 typedef struct {
2     int id;
3     int remainingTime;
4 } Vehicle;
5
6 typedef struct {
7     Vehicle data[MAX];
8     int front, rear;
9 } Queue;
10
11 void initQueue(Queue *q) { q->front = q->rear = -1; }
12
13 int isEmpty(Queue *q) { return (q->front == -1); }
14
15 void enqueue(Queue *q, Vehicle v) {
16     if (q->rear == MAX - 1) return;
17     if (q->front == -1) q->front = 0;
18     q->data[++q->rear] = v;
19 }
20
21 Vehicle dequeue(Queue *q) {
22     Vehicle temp; temp.id = -1;
23     if (isEmpty(q)) return temp;
24     temp = q->data[q->front];
25     if (q->front == q->rear) q->front = q->rear = -1;
26     else q->front++;
27     return temp;
28 }

```

Listing 2: Queue struct and core operations

7.2. B. Signal Cycle Function

```

1 void signalCycle() {
2     printf("\n RED      - STOP      (5 sec)\n"); Sleep(5000);
3     printf("  YELLOW - PREPARE (5 sec)\n"); Sleep(5000);
4     printf("  GREEN  - GO        (45 sec)\n");
5 }

```

Listing 3: Signal cycle with real-time delays

7.3. C. FCFS Core Loop

```

1 signalCycle();
2 DWORD startTime = GetTickCount();
3 while ((GetTickCount() - startTime < 45000) && !isEmpty(q)) {
4     Vehicle v = dequeue(q);
5     printf("  Vehicle %d crossing...\n", v.id);

```

```
6     Sleep(v.remainingTime * 1000);
7     printf("  Vehicle %d crossed\n", v.id);
8 }
```

Listing 4: FCFS green-phase dispatch loop

7.4. D. Round Robin Core Loop

```
1 signalCycle();
2 DWORD startTime = GetTickCount();
3 while ((GetTickCount() - startTime < 45000) && !isEmpty(q)) {
4     Vehicle v = dequeue(q);
5     if (v.remainingTime > timeQuantum) {
6         Sleep(timeQuantum * 1000);
7         v.remainingTime -= timeQuantum;
8         enqueue(q, v);          /* partial -- re-enqueue */
9     } else {
10        Sleep(v.remainingTime * 1000);
11        printf("  Vehicle %d completed\n", v.id);
12    }
13 }
```

Listing 5: Round Robin green-phase dispatch loop

7.5. E. Compilation Instructions

```
# Windows (MinGW):
gcc -o traffic_signal traffic_signal.c -lwinmm -Wall
```

```
# Run:
traffic_signal.exe
```

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. [CPU scheduling chapter – FCFS and Round Robin algorithms.]
3. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
4. Microsoft Docs. *GetTickCount function (sysinfoapi.h)*. <https://docs.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-gettickcount>
5. GeeksforGeeks. *Queue Data Structure*. <https://www.geeksforgeeks.org/queue-data-structure/>
6. GeeksforGeeks. *Round Robin Scheduling Algorithm*. <https://www.geeksforgeeks.org/program-round-robin-scheduling-set-1/>